Πετράκης Ραφαήλ Ιωάννης - csd5748

ΗΥ-240 - Δομές Δεδομένων - Άσκηση #1

· Άσκηση 1η - Ερώτημα 1ο:

procedure $PuzzleA$(integer n)
  for $(i = 1;\ \ i \le n;\ \ i++)$ $\longrightarrow$ $i = 1 \ldots i = n \Rightarrow n$ *loops*.
    for $(j = 1;\ \ j \le i;\ \ j++)$ $\longrightarrow$ $j = 1 \ldots j = i \Rightarrow i$ *loops*.
      for $(k = 1;\ \ k \le j;\ \ k++)$ $\longrightarrow$ $k = 1 \ldots k = j \Rightarrow j$ *loops*.
        $z = z + 1$;

Έτσι έχουμε: $$\sum_{i=1}^{n}\left(\sum_{j=1}^{i}\left(\sum_{k=1}^{j}\left(1\right)\right)\right) \;=\; \sum_{i=1}^{n}\sum_{j=1}^{i}\sum_{k=1}^{j}1$$

Και: $$\sum_{k=1}^{j}1 = j \;\Rightarrow\; \sum_{i=1}^{n}\sum_{j=1}^{i} j$$

Και: $$\sum_{j=1}^{i} j = \tfrac{i(i+1)}{2} \;\Rightarrow\; \sum_{i=1}^{n}\frac{i(i+1)}{2}$$

Και: $$\sum_{i=1}^{n}\frac{i(i+1)}{2} \;=\; \frac{1}{2}\sum_{i=1}^{n}(i^2+i) \;=\; \frac{1}{2}\left(\sum_{i=1}^{n} i^2 + \sum_{i=1}^{n} i\right)$$

Και: $$\sum_{i=1}^{n} i \;=\; \frac{n(n+1)}{2} \;\;\&\;\; \sum_{i=1}^{n} i^2 \;=\; \frac{n(n+1)(2n+1)}{6} \;\Rightarrow\; \frac{1}{2}\left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2}\right) \;=\; \frac{n(n+1)(n+2)}{6}$$

Τελικά: $$\boxed{T_1(n) \;=\; \frac{n(n+1)(n+2)}{6}}$$

· Άσκηση 1η - Ερώτημα 2ο:

procedure $PuzzleB$(integer n)
  for $(i = 1;\ \ i \le n;\ \ i++)$ $\longrightarrow$ $i = 1 \ldots i = n \Rightarrow n$ *loops*.
    if $(i\ \%\ 2 == 1)$ $\longrightarrow$ $i == 2k+1 \;\Rightarrow\; k \in [\,1,\ \lfloor (n-1)/2 \rfloor\,]$ *loops*.
      for $(j = 1;\ \ j \le i;\ \ j++)$ $\longrightarrow$ $j = 1 \ldots j = i \Rightarrow i$ *loops*.
        $z = z + 1$;

Έτσι έχουμε: $$\sum_{i=1}^{n}\Big(\sum_{j=1}^{(i\ \%\ 2 == 1)}(1)\Big) \;=\; \sum_{\substack{i=1 \\ (i\ odd)}}^{n} i \;=\; 1 + 2 + \ \ldots\ + (2k+1) \;\overset{*}{=}\; (k+1)^2.$$

Τελικά: $$\boxed{T_2(n) \;=\; (\lfloor \frac{n-1}{2} \rfloor + 1)^2} \quad (\text{αφού } max_k = \lfloor (n-1)/2 \rfloor \ \ldots)$$

· Άσκηση 1η - Ερώτημα 3ο:

procedure $PuzzleC(\text{integer } n)$
  for $(i = 1;\ \ i \le n;\ \ i++)$ $\longrightarrow$ $i = 1 \ldots i = n \Rightarrow n$ *loops*.
    for $(j = 1;\ \ j \le n * i;\ \ j++)$ $\longrightarrow$ $j = 1 \ldots j = n \cdot i \Rightarrow n \cdot i$ *loops*.
      $z = z + 1;$

Έτσι έχουμε: $\sum_{i=1}^{n}(\sum_{j=1}^{n*i}(1)) \ = \ \sum_{i=1}^{n}(ni) \ = \ n * \sum_{i=1}^{n} i \ = \ n \cdot \frac{n(n+1)}{2}$

Τελικά: $\boxed{T_3(n) = \frac{n^2(n+1)}{2}}$

· Άσκηση 2η - Ερώτημα 1ο:

Έχουμε: $F(n) = 5n^3 + 2n^2 \log n - 3n \log_2 n + 7$

Γνωρίζουμε: $n^3 \gg n^2 \log n \gg n \log n \gg 1$

Έτσι: $\boxed{F(n) = \Theta(n^3)}$ (η ασυμπτωτική πολυπλοκότητα βάσει συμβολισμού Θ)

· Άσκηση 2η - Ερώτημα 2ο:

Έχουμε: $n \log n, \log^2 n, n^{1/2}, 2^{\log n}, n^3, n^2 \log^5 n, \log(n!), n^{\log n}, 3^n, n^{1.5}, \log n, 2^n, n, n^2, n + 1$

$g_1 = 3^n$	$\Theta_{10} : \{3^n\}$
$g_2 = 2^n$	$\Theta_9 : \{2^n\}$
$g_3 = n^{\log n}$	$\Theta_8 : \{n^{\log n}\}$
$g_4 = n^3$	$\Theta_7 : \{n^3\}$
$g_5 = n^2 \log^5 n$	$\Theta_6 : \{n^2, n^2 \log^5 n\}$
$g_6 = n^2$	$\Theta_6 : \{n^2, n^2 \log^5 n\}$
$g_7 = n^{1.5}$	$\Theta_5 : \{n^{1.5}\}$
$g_8 = n \log n$	$\Theta_4 : \{n \log n, \log(n!)\}$
$g_9 = \log(n!)$	$\Theta_4 : \{n \log n, \log(n!)\}$
$g_{10} = n$	$\Theta_3 : \{n, n + 1, 2^{\log n}\}$
$g_{11} = n + 1$	$\Theta_3 : \{n, n + 1, 2^{\log n}\}$
$g_{12} = 2^{\log n}$	$\Theta_3 : \{n, n + 1, 2^{\log n}\}$
$g_{13} = n^{1/2}$	$\Theta_2 : \{n^{1/2}\}$
$g_{14} = \log^2 n$	$\Theta_1 : \{\log n, \log^2 n\}$
$g_{15} = \log n$	$\Theta_1 : \{\log n, \log^2 n\}$

· Άσκηση 3η - Ερώτημα 1ο:

Έχουμε: $T(n) = \begin{cases} 1, & n = 1 \\ 3\,T(\lfloor n/2 \rfloor) + n^2, & n \neq 1 \end{cases}$

Επαναληπτική αντικατάσταση (αναδρομή): $T(n) = 3 * T(n/2) + n^2$

Αφού: $T(n/2) = 3 * T(n/4) + (n/2)^2 \;\Rightarrow\; T(n) = 3\big(3 * T(n/4) + (n/2)^2\big) + n^2 \;=\; 9 * T(n/4) + \frac{7}{4}n^2$

Επίσης: $T(n/4) = 3 * T(n/8) + (n/4)^2 \;\Rightarrow\; T(n) = 9\big(3 * T(n/8) + (n/4)^2\big) + \frac{7}{4}n^2 \;=\; 27 * T(n/8) + \frac{37}{16}n^2$

Μοτίβο (μετά από i επαναλήψεις): $T(n) = 3^i * T\left(\frac{n}{2^i}\right) + n^2 \cdot \sum_{k=0}^{(i-1)} 3^k \left(\frac{1}{2^{2k}}\right)$

Σταματάμε όταν: $\frac{n}{2^i} = 1 \implies i = \log_2(n)$

Για $i = \log n$: $T(n) = 3^{\log(n)} \cdot T(1) + n^2 \sum_{k=0}^{(\log(n-1))} \left(\left(\frac{3}{4}\right)^k\right) \implies T(n) = n^{\log 3} + n^2 \cdot \sum_{k=0}^{\log(n-1)} \left(\frac{3}{4}\right)^k$

Και: $\sum_{k=0}^{\log(n-1)} \left(\frac{3}{4}\right)^k \underset{*}{\overset{\text{Ιδ.}}{=}} \frac{1-(3/4)^{\log n}}{1-3/4} = 4 \cdot \left[1 - \left(\frac{3}{4}\right)^{\log n}\right] = 4\left[1 - \frac{n^{\log_2 3}}{n^2}\right] = 4 - 4 \cdot \frac{n^{\log_2 3}}{n^2}$

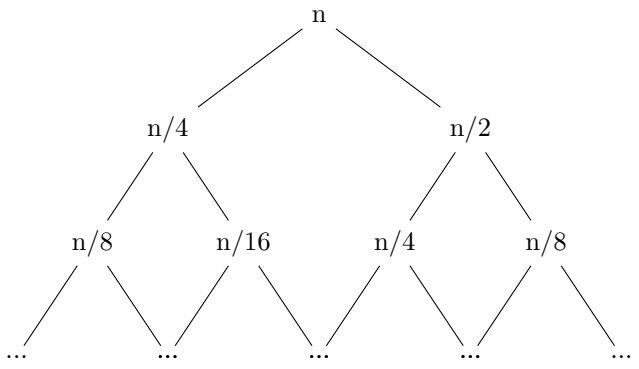
Οπότε: $T(n) = n^{\log_2 3} + n^2 \cdot \left[4 - 4 \cdot \frac{n^{\log_2 3}}{n^2}\right] = n^{\log_2 3} + (4n^2 - 4n^{\log_2 3}) = 4n^2 - 3n^{\log_2 3}$

Τελικά, Κλειστός Τύπος: $T(n) = n^{\log_2 3} + (4n^2 - 4n^{\log_2 3}) = 4n^2 - 3n^{\log_2 3}$

Και, Ασυμπτωτικό Φράγμα: $T(n) = O(n^2)$ αφού για $n \longrightarrow \infty :\; n^2 > n^{\log_2 3}$.

· Άσκηση 3η - Ερώτημα 2ο:

Έχουμε: $T(n) = T(n/4) + T(n/2) + n$



Ασυμπτωτική συμπεριφορά: $cost(level_1) = n,\; cost(level_2) = n/4 + n/2 = 3n/4,\; cost(level_3) = 9n/16, \quad ...$

Άρα: $\Theta(n)$ (ύψος δέντρου $O(logn)$)

· Άσκηση 4η - Ερώτημα 1ο:

```
1: Pseudo-Function FindMax(A, i, j):
2: if (i = j) then
3:    return (A[i])
4: else
5:    Middle ← ⌊(i + j)/2⌋
6:    Left_max ← FindMax(A, i, Middle)
7:    Right_max ← FindMax(A, Middle+1, j)
8:    if (Left_max > Right_max) then
9:       return (Left_max)
10:   else
11:      return (Right_max)
12:   end if
13: end if=0
```

· Άσκηση 3η - Ερώτημα 2ο:

Έχουμε: $T(n) = ?$ για $n = j - i + 1$ (αφού (υπο)πίνακας Α δισδιάστατος ...)

Θα είναι: $T(n) = \begin{cases} c, & n = 1, \\ 2 \cdot T(n/2) + c, & n > 1. \end{cases}$

· Άσκηση 3η - Ερώτημα 3ο:

Ομοίως με άσκ3: $\implies$ ... $\implies$ $T(n) = \Theta(n) \Rightarrow T(n) = O(n) = \Omega(n)$ ...

· Άσκηση 3η - Ερώτημα 4ο:

Ιχνηλάτηση του FindMax([4,7,1,9,2], 0, 4):

Κλήση 1: FindMax(A, 0, 4) − Παράμετροι : $i = 0, j = 4, n = j - i + 1 = 5$.

Επειδή i $\neq$ j, υπολογίζουμε το μέσο σημείο: middle $= \left\lfloor \dfrac{i+j}{2} \right\rfloor = \left\lfloor \dfrac{0+4}{2} \right\rfloor = 2$

Αναδρομική κλήση 1 : $Left_{max} =$ FindMax(A, 0, 2) (αριστερό τμήμα : $[4, 7, 1]$)

Αναδρομική κλήση 2 : $Right_{max} =$ FindMax(A, 3, 4) (δεξί τμήμα : $[9, 2]$)

Κλήση 2: FindMax(A, 0, 2) − Παράμετροι : $i = 0, j = 2, n = j - i + 1 = 3$.

Επειδή i $\neq$ j, υπολογίζουμε το μέσο σημείο: middle $= \left\lfloor \dfrac{i+j}{2} \right\rfloor = \left\lfloor \dfrac{0+2}{2} \right\rfloor = 1$

Αναδρομική κλήση 3 : $Left_{max} =$ FindMax(A, 0, 1) (αριστερό τμήμα : $[4, 7]$)

Αναδρομική κλήση 4 : $Right_{max} = \text{FindMax(A, 2, 2)}$ (δεξί τμήμα : [1])

Κλήση 3: FindMax(A, 0, 1) − Παράμετροι : $i = 0, j = 1, n = j - i + 1 = 2$.

Επειδή i $\neq$ j, υπολογίζουμε το μέσο σημείο: $\text{middle} = \left\lfloor \frac{i+j}{2} \right\rfloor = \left\lfloor \frac{0+1}{2} \right\rfloor = 0$

Αναδρομική κλήση 5 : $Left_{max} = \text{FindMax(A, 0, 0)}$ (αριστερό τμήμα : [4])

Αναδρομική κλήση 6 : $Right_{max} = \text{FindMax(A, 1, 1)}$ (δεξί τμήμα : [7])

Κλήση 4: FindMax(A, 0, 0) − Παράμετροι : $i = 0, j = 0, n = 1$.

Βάση αναδρομής, οπότε επιστρέφει: $A[0] = 4$

Κλήση 5: FindMax(A, 1, 1) − Παράμετροι : $i = 1, j = 1, n = 1$.

Βάση αναδρομής, οπότε επιστρέφει: $A[1] = 7$

Επιστροφή στην Κλήση 3: FindMax(A, 0, 1) − Σύγκριση : $Left_{max} = 4, Right_{max} = 7$.

Επιστρέφει: $\max(4, 7) = 7$

Κλήση 6: FindMax(A, 2, 2) − Παράμετροι : $i = 2, j = 2, n = 1$.

Βάση αναδρομής, οπότε επιστρέφει: $A[2] = 1$

Επιστροφή στην Κλήση 2: FindMax(A, 0, 2) − Σύγκριση : $Left_{max} = 7, Right_{max} = 1$.

Επιστρέφει: $\max(7, 1) = 7$

Κλήση 7: FindMax(A, 3, 4) − Παράμετροι : $i = 3, j = 4, n = j - i + 1 = 2$.

Επειδή i $\neq$ j, υπολογίζουμε το μέσο σημείο: $\text{middle} = \left\lfloor \frac{i+j}{2} \right\rfloor = \left\lfloor \frac{3+4}{2} \right\rfloor = 3$

Αναδρομική κλήση 8 : $Left_{max} = \text{FindMax(A, 3, 3)}$ (αριστερό τμήμα : [9])

Αναδρομική κλήση 9 : $Right_{max} = \text{FindMax(A, 4, 4)}$ (δεξί τμήμα : [2])

Κλήση 8: FindMax(A, 3, 3) − Παράμετροι : $i = 3, j = 3, n = 1$.

Βάση αναδρομής, οπότε επιστρέφει: $A[3] = 9$

Κλήση 9: FindMax(A, 4, 4) − Παράμετροι : $i = 4, j = 4, n = 1$.

Βάση αναδρομής, οπότε επιστρέφει: $A[4] = 2$

Επιστροφή στην Κλήση 7: FindMax(A, 3, 4) − Σύγκριση : $Left_{max} = 9, Right_{max} = 2$.

Επιστρέφει: $\max(9, 2) = 9$

Επιστροφή στην Κλήση 1: FindMax(A, 0, 4) − Σύγκριση : $Left_{max} = 7, Right_{max} = 9$.

Επιστρέφει: $\max(7, 9) = 9$

Τελικό αποτέλεσμα: Το μέγιστο στοιχείο του πίνακα είναι 9.

· Άσκηση 3η - Ερώτημα 5ο:

Χωρική πολυπλοκότητα αλγορίθμου:

−Τοπικές μεταβλητές: $i, j, \text{middle}, Left_{max}, Right_{max}$ καταλαμβάνουν σταθερό χώρο $O(1)$ ανά κλήση.

−Αναδρομική στοίβα: Το βάθος της αναδρομής είναι $\log_2(n)$, καθώς ο πίνακας διαιρείται στη μέση σε κάθε κλήση.

−Χώρος στοίβας: $\log_2(n)$ στοιχεία, με κάθε στοιχείο $O(1)$. Άρα, $O()$.

−Είσοδος: Ο πίνακας A δεν προσμετράται, καθώς δεν δημιουργούνται νέες δομές δεδομένων.

Τελικά, συνολική χωρική πολυπλοκότητα: $O(\log n)$.

Πετράκης Ραφαήλ Ιωάννης - csd5748

ΗΥ-240 - Δομές Δεδομένων - Ασκήσεις #2

⋆ **Άσκηση 1η - Ερώτημα α:**

Βήμα	Είσοδος	Ουρά (μπροστά → πίσω)	Ενέργεια	Επιστροφή
1	A	A	enqueue(A)	-
2	B	A B	enqueue(B)	-
3	C	A B C	enqueue(C)	-
4	*	B C	dequeue()	A
5	D	B C D	enqueue(D)	-
6	E	B C D E	enqueue(E)	-
7	*	C D E	dequeue()	B
8	*	D E	dequeue()	C
9	F	D E F	enqueue(F)	-
10	G	D E F G	enqueue(G)	-
11	*	E F G	dequeue()	D
12	*	F G	dequeue()	E
13	H	F G H	enqueue(H)	-
14	*	G H	dequeue()	F
15	*	H	dequeue()	G

⋆ Εκτέλεση της ακολουθίας (A B C * D E * * F G * * H * *) σε κενή ουρά ...

Τελικά: $\boxed{A,\ B,\ C,\ D,\ E,\ F,\ G}$

⋆ **Άσκηση 1η - Ερώτημα β:**

(i) Algorithm Find(L, k):
```
current ← L.head
while current != null
   if current.data = k
      return current
   current ← current.next
return null
```

(ii) Algorithm InsertAfter(L, k1, k2):
```
current ← Find(L, k1)
if current = null
    return
newnode ← new node
newnode.data ← k2
newnode.next ← current.next
newnode.prev ← current
if current.next != null
    current.next.prev ← newNode
else
    L.tail ← newNode
current.next ← newNode
```

(iii) Algorithm DeleteValue(L, k):
```
current ← Find(L, k)
if current = null return
if current.prev != null
    current.prev.next ← current.next
else
    L.head ← current.next
if current.next != null
    current.next.prev ← current.prev
else
    L.tail ← current.prev
free(current)
```

⋆ **Άσκηση 2η - Ερώτημα α:**

```
Function TransferAlternate(Stack S, Queue Q):
    Stack R = EmptyStack()
    turn ← "S"
    while (not IsEmpty(S)) or (not IsEmpty(Q))
      if (turn = "S") and (not IsEmpty(S))
        x ← Pop(S)
        Push(R, x)
        turn ← "Q"
      else if (turn = "Q") and (not IsEmpty(Q))
        x ← Dequeue(Q)
        Push(R, x)
        turn ← "S"
      else if (IsEmpty(S)) and (not IsEmpty(Q))
        x ← Dequeue(Q)
        Push(R, x)
      else if (IsEmpty(Q)) and not (IsEmpty(S))
        x ← Pop(S)
        Push(R, x)
    return (R)
```

⋆ **Άσκηση 2η - Ερώτημα β:**

```
Function IsBalanced(StringSequence):
    Stack S = EmptyStack()
    for each character in StringSequence:
      if (character = "(") or (character = "[") or (character = "{"):
        Push(S, character)
      else if (character = ")") or (character = "]") or (character = "}"):
        if (IsEmpty(S)): return (False)
        top ← Pop(S)
        if (character = ")" and top != "(") or (character = "]" and top != "[") or (character = "}" and top != "{"):
          return (False)
    if (IsEmpty(S)):
      return (True)
    else:
      return (False)
```

⋆ **Άσκηση 3η - Ερώτημα α:**

```
Algorithm MergeUnique(A, B):
    List C = EmptyList();
    pointerA ← A.head;
    pointerB ← B.head;
    LastValue ← None;
    while (pointerA != null) and (pointerB != null):
      if (pointerA.data ¡ pointerB.data):
        value ← pointerA.data;
        pointerA ← pointerA.next;
      else if (pointerA.data ¿ pointerB.data):
        value ← pointerB.data;
        pointerB ← pointerB.next;
      else:
        value ← pointerA.data;
        pointerA ← pointerA.next;
        pointerB ← pointerB.next;
      if (value != LastValue):
        Append(C, value);
        LastValue ← value;
    while (pointerA != null):
      value ← pointerA.data;
      pointerA ← pointerA.next;
      if (value != LastValue):
        Append(C, value);
        LastValue ← value;
    while (pointerB != null):
      value ← pointerB.data;
      pointerB ← pointerB.next;
      if (value != LastValue):
        Append(C, value);
        LastValue ← value;
    return (C);
```

⋆ **Άσκηση 3η - Ερώτημα β:**

```
Algorithm ReverseCopy(L):
    if (L = Null): return Null;
    NewHead ← ReverseCopy(L.next);
    NewNode ← new Node;
    NewNode.data ← L.data;
    NewNode.next ← NewHead;
    return NewNode;
```


LaTeX

Πετράκης Ραφαήλ Ιωάννης ⋆ csd5748

ΗΥ-240 - Δομές Δεδομένων - Ασκήσεις #3

⋆ Άσκηση 1η - Ερώτημα α:

```
function isPerfect(root):
    if root == NULL: return TRUE
    h = height(root)
    return isPerfectRec(root, 0, h)

function isPerfectRec(node, level, h):
    if node == NULL: return TRUE
    if node.left == NULL and node.right == NULL:
       return level + 1 == h
    if node.left == NULL or node.right == NULL:
       return FALSE
    return isPerfectRec(node.left, level+1, h) and isPerfectRec(node.right, level+1, h)
```

⋆ Άσκηση 1η - Ερώτημα β:

```
function isAlmostComplete(root, h):
    if root == NULL: return h == 0
    if height(root) != h: return FALSE
    n = countNodes(root)
    if n < 2^(h−1) or n > 2^h - 1: return FALSE
    return checkSeq(root, 0, n)

function checkSeq(node, idx, n):
    if node == NULL: return TRUE
    if idx >= n: return FALSE
    return checkSeq(node.left, 2∗idx+1, n) and checkSeq(node.right, 2∗idx+2, n)
```

⋆ **Άσκηση 2η - Ερώτημα α:**

```
function splitBST(root, K) → (L, R):
    if root == NULL: return (NULL, NULL)
    if root.data < K:
       (leftSub, rightSub) = splitBST(root.right, K)
       root.right = leftSub
       return (root, rightSub)
    else:
       (leftSub, rightSub) = splitBST(root.left, K)
       root.left = rightSub
       return (leftSub, root)
```

⋆ **Άσκηση 2η - Ερώτημα β:**

```
mergeBST(T1, T2) → T:
    if T1 == NULL return T2
    if T2 == NULL return T1
    L1 = inOrderToList(T1)          // O(n1).
    L2 = inOrderToList(T2)          // O(n2).
    L = mergeSortedLists(L1, L2)    // O(n1 + n2).
    return listToBalancedBST(L)     // O(n1 + n2).
```

⋆ **Άσκηση 3η:**

```
function Delete(T, K) → new_root:
    if T == NULL: return NULL
    toDelete = BFS_find_node(T, K)
    if toDelete == NULL: return T
    last = BFS_last_node(T)
    if toDelete == last:
      remove_leaf(toDelete)
      if toDelete was root: T = NULL
    else:
      toDelete.key = last.key
      remove_leaf(last)
    return T

function BFS_find_node(root, K) → Node:
    queue = [root]
    while queue not empty:
      curr = dequeue()
      if curr.key == K: return curr
      if curr.left: enqueue(curr.left)
      if curr.right: enqueue(curr.right)
    return NULL

function BFS_last_node(root) → Node:
    queue = [root]
    last = NULL
    while queue not empty:
      last = dequeue()
      if last.left: enqueue(last.left)
      if last.right: enqueue(last.right)
    return last

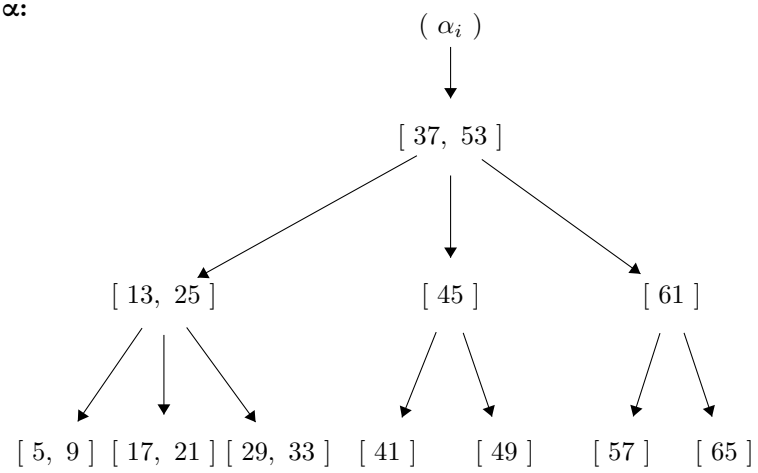
// O(n) != O(logn) ...
```

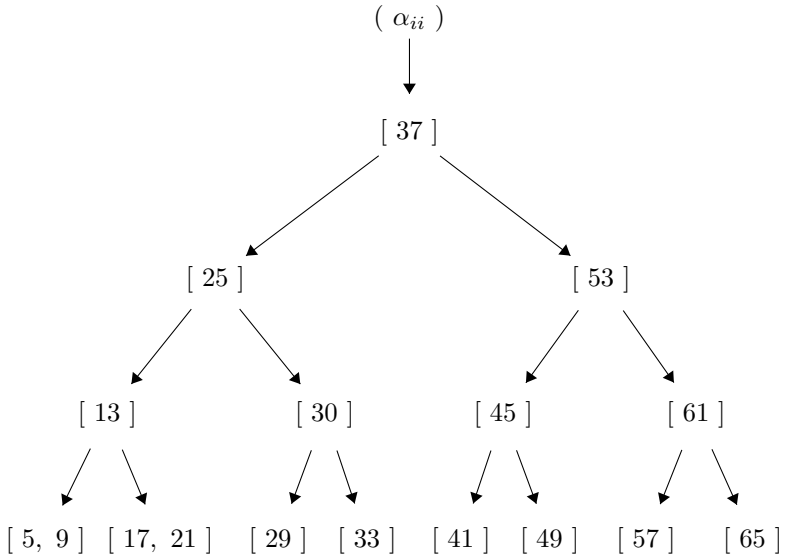

LaTeX

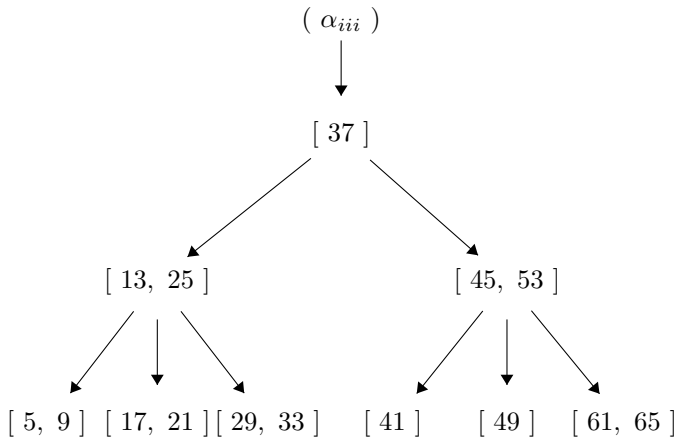
Πετράκης Ραφαήλ Ιωάννης ⋆ csd5748

ΗΥ-240 - Δομές Δεδομένων - Ασκήσεις #4

⋆ Άσκηση 1η - Ερώτημα α:







⋆ **Άσκηση 1η - Ερώτημα β:**

β_i)
Union(3,0), Union(3,4), Union(3,8),
Union(7,1), Union(7,5), Union(7,6), Union(7,9),
Union(14,2), Union(14,10), Union(14,11), Union(14,13),
Union(12,3), Union(12,7), Union(12,14), Union(12,15)

β_{ii})
Δεν είναι μοναδική, αφού η σείρα των συνδέσεων κόμβων αλλά και υποδέντρων δεν θα αλλάξει το τελικό αποτέλεσμα ...

⋆ **Άσκηση 2η - Ερώτημα α:**

i) Πεδίο *augment*: πλήθος (*count*) κόμβων στο υποδένδρο του κάθε κόμβου με ρίζα τον κόμβο αυτόν (T) ...
ii) Ενημέρωση: πλήθος = 1 + πλήθος αριστερού κόμβου + πλήθος δεξιού κόμβου (αν κόμβος = *null*: πλήθος = 0) ...
iii) Αλγόριθμος (Ψευδοκώδικας):

```
function RangeCount(T, a, b):
    if T = null:
        return 0;
    count = 0
    if T.key >= a and T.key <= b:
        count = count + 1 + left.size + RangeCount(T.right, a, b)
    else if T.key < a:
        count = count + RangeCount(T.right, a, b)
    else
        count = count + RangeCount(T.left, a, b)
    return count;
```

⋆ **Άσκηση 2η - Ερώτημα β:**

i) Πεδίο *augment*: δεν χρειάζεται να αποθηκεύσουμε τίποτα ...
ii) Ενημέρωση: δεν χρειάζεται να ενημερώσουμε τίποτα ...
iii) Αλγόριθμος (Ψευδοκώδικας):

```
function PrefixMax(T, k):
    result = null
    current = T
    while current != null:
        if current.key == k:
            return current.key;
        else if current.key < k:
            result = current.key
            current = current.right
        else
            current = current.left
    return result;
```

⋆ **Άσκηση 2η - Ερώτημα γ:**

i) Πεδίο *augment*: μέγεθος υποδένδρου (*size*) & άθροισμα όλων των κλειδιών στο υποδένδρο του T (*sum*) ...
ii) Ενημέρωση: μέγεθος υποδένδρου - ομοίως με πριν & άθροισμα = κλειδί κόμβου + άθροισμα αριστερού + άθροισμα δεξιού ...
iii) Αλγόριθμος (Ψευδοκώδικας):

```
function KthSum(T, k):
    if T = null or k <= 0:
        return 0;
    leftSize = (T.left != null) ? T.left.size : 0
    leftSum = (T.left != null) ? T.left.sum : 0
    if k <= leftSize:
        return KthSum(T.left, k);
    else if k == leftSize + 1:
        return leftSum + T.key;
    else:
        return leftSum + T.key + KthSum(T.right, k - leftSize - 1);
```

⋆ **Άσκηση 3η - Ερώτημα α:**

```
function FindMinKeyWithRedDepth(T, k):
    depth = 0
    while T != null:
        if depth == k:
            return T.key;
        is_red = (T.color == RED) ? 1 : 0
        next_depth = depth + is_red
        if T.left != null && next_depth + T.left.max_red_path >= k:
            T = T.left
        else:
            T = T.right
        depth = next_depth
    return null;
```

⋆ **Άσκηση 3η - Ερώτημα β:**

```
function CanBeRBTree(r):
    if r = null:
        return true;
    function CheckUniformLeafDepth(v):
        if v = null:
            return 0;
        left_h = CheckUniformLeafDepth(v.left)
        right_h = CheckUniformLeafDepth(v.right)
        if left_h == -1 || right_h == -1:
            return -1;
        if left_h != right_h:
            return -1;
        return left_h + 1;
    return (CheckUniformLeafDepth(r) != -1);
```


LaTeX